

机器学习

2.1 决策树

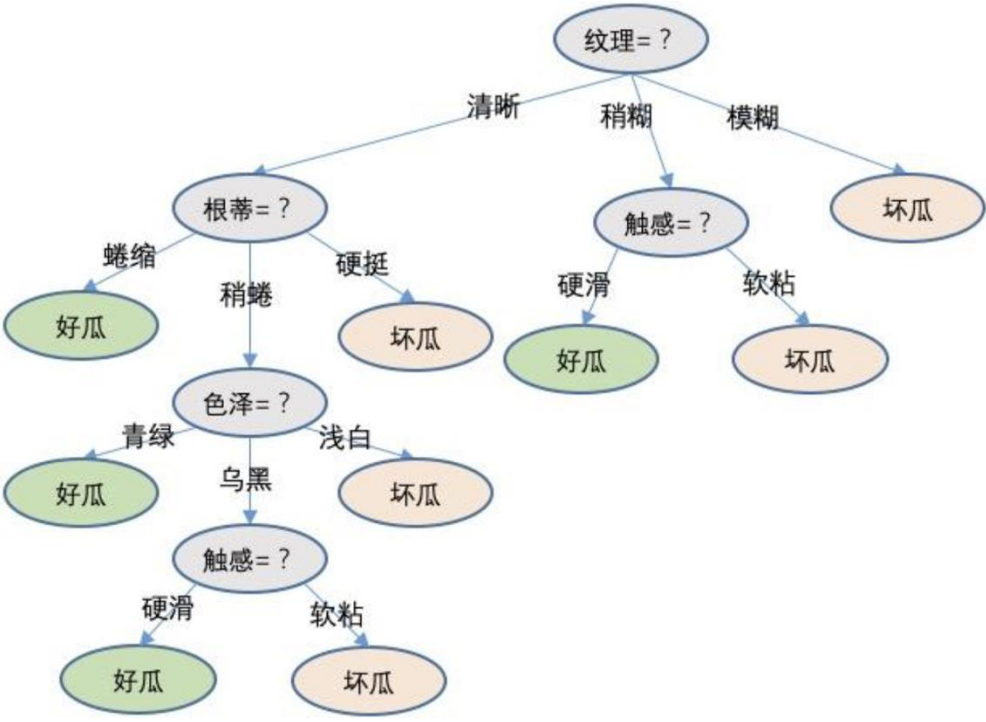
吕斌

上海外国语大学国际金融贸易学院

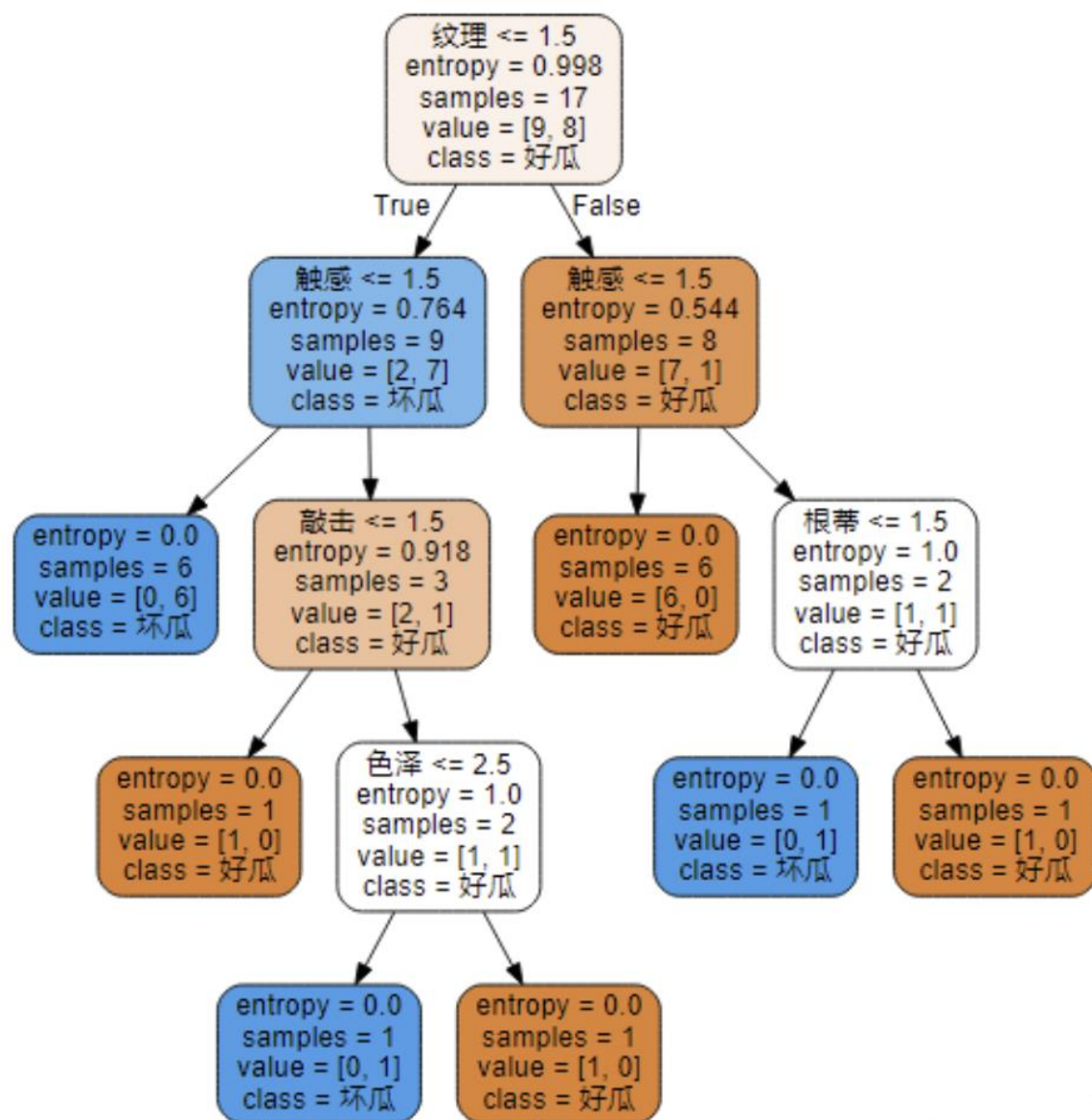
决策树

- 决策树（Decision Tree）是一种非参数的有监督学习方法，它能够从一系列有特征和标签的数据中总结出决策规则，并用树状图的结构来呈现这些规则，以解决分类和回归问题。决策树算法容易理解，适用各种数据，在解决各种问题时都有良好表现，尤其是以树模型为核心的各种集成算法，在各行各业和领域都有广泛的应用。决策树算法的本质是一种图结构，只需要问一系列问题就可以对数据进行分类。

	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
10	青绿	硬挺	清脆	清晰	平坦	软粘	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	否

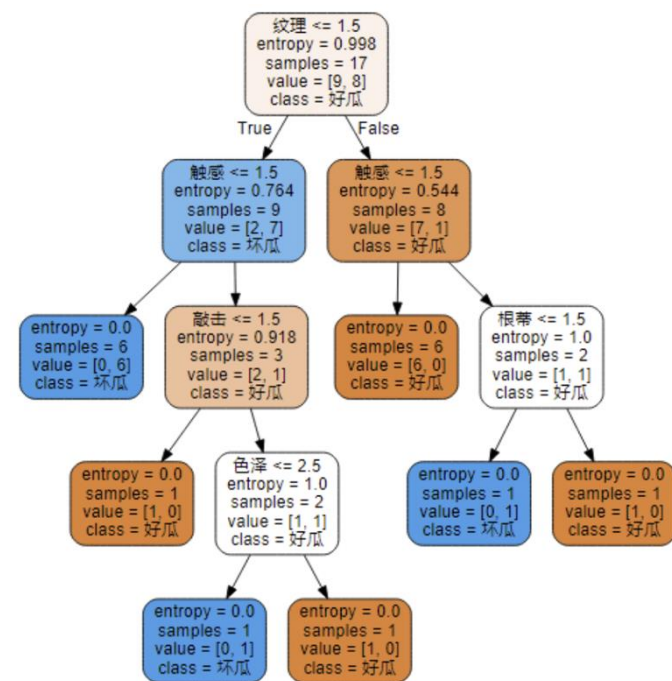


决策树



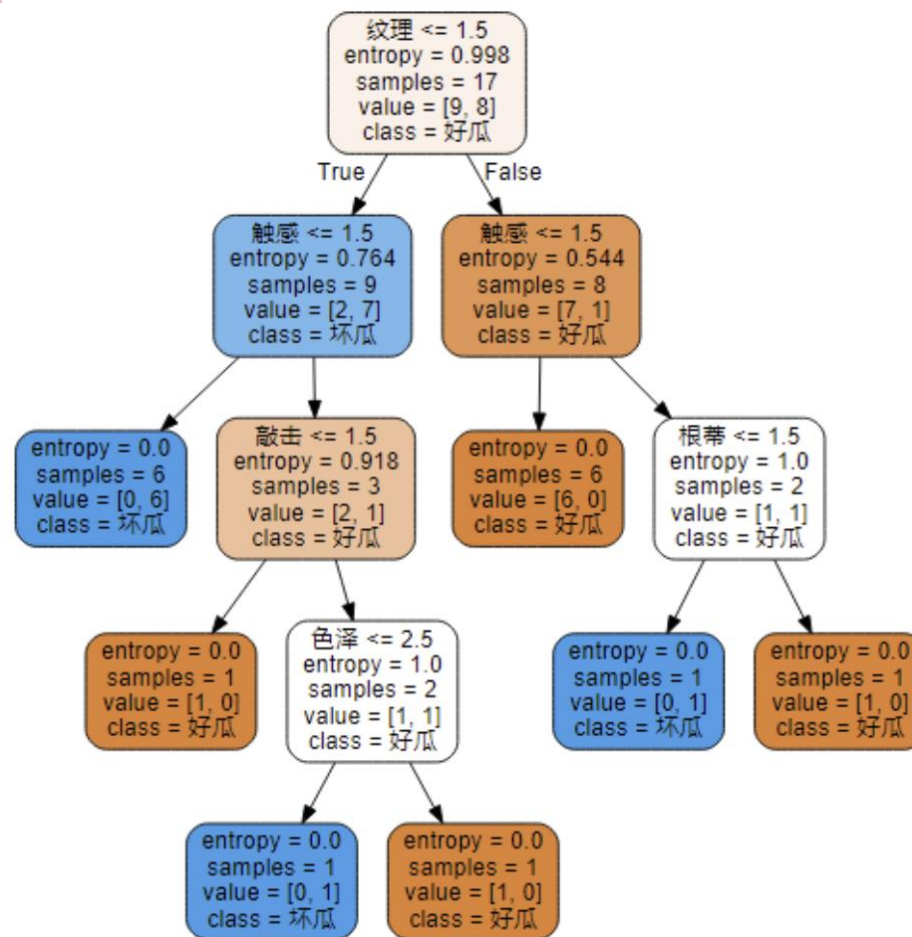
决策树

- 可以看出，在决策过程中，一直在对记录的特征进行提问。最初的问题所在的地方叫做根节点，在得到结论前的每一个问题都是中间节点，而得到的每一个结论（动物的类别）都叫做叶子节点。
 - 根节点：没有进边，有出边。包含最初的，针对特征的提问。
 - 中间节点：既有进边也有出边，进边只有一条，出边可以有很多条。都是针对特征的提问。
 - 叶子节点：有进边，没有出边，每个叶子节点都是一个类别标签。
 - 子节点和父节点：在两个相连的节点中，更接近根节点的是父节点，另一个是子节点。



决策树需要解决的核心问题

- 1、如何从数据表中找出最佳节点和最佳分枝？
 - 不纯度的下降，即信息增益比例增加，怎么衡量“增加”？
 - 决策树有ID3、C4.5、CART等不同类型
 - ID3算法计算的是信息增益
 - C4.5算法计算的是信息增益比率
 - CART算法通过基尼系数代替信息增益算法
- 2、如何让决策树停止生长，防止过拟合？
 - 剪枝参数、控制分裂参数等



```
class sklearn.tree.DecisionTreeClassifier (criterion='gini', splitter='best', max_depth=None,  
min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=None,  
random_state=None, max_leaf_nodes=None, min_impurity_decrease=0.0, min_impurity_split=None,  
class_weight=None, presort=False)
```

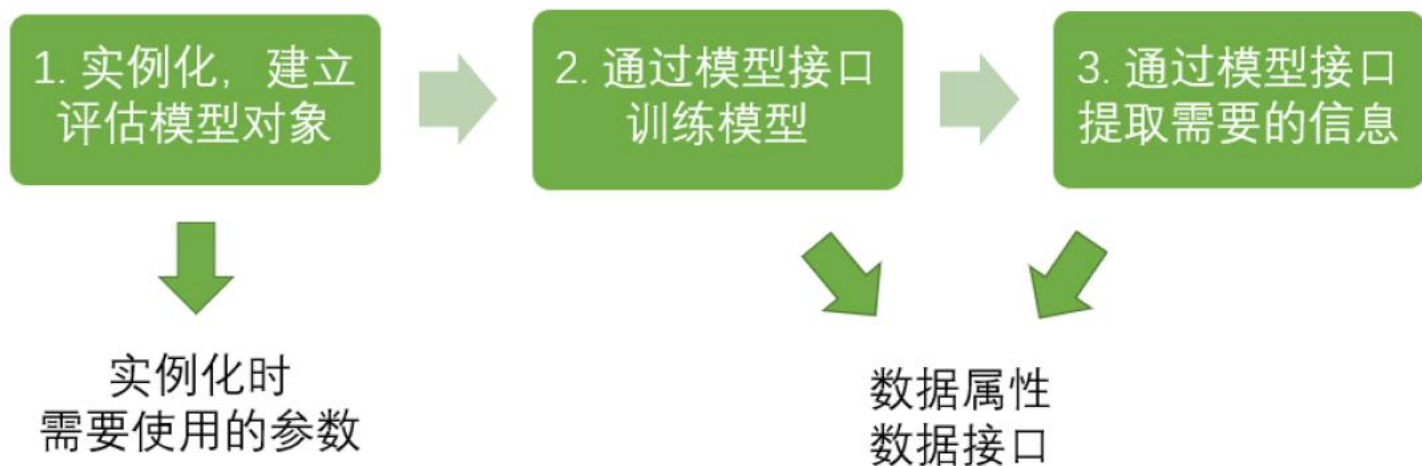
Sklearn中的决策树

- 模块sklearn.tree
 - sklearn中决策树的类都在“tree”模块之下。这个模块总共包含五个类：

<code>tree.DecisionTreeClassifier</code>	分类树
<code>tree.DecisionTreeRegressor</code>	回归树
<code>tree.export_graphviz</code>	将生成的决策树导出为DOT格式，画图专用
<code>tree.ExtraTreeClassifier</code>	高随机版本的分类树
<code>tree.ExtraTreeRegressor</code>	高随机版本的回归树

```
class sklearn.tree.DecisionTreeClassifier (criterion='gini', splitter='best', max_depth=None,  
min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=None,  
random_state=None, max_leaf_nodes=None, min_impurity_decrease=0.0, min_impurity_split=None,  
class_weight=None, presort=False)
```

Sklearn基本建模流程



```
from sklearn import tree
```

#导入需要的模块

```
clf = tree.DecisionTreeClassifier()
```

#实例化

```
clf = clf.fit(X_train,y_train)
```

#用训练集数据训练模型

```
result = clf.score(X_test,y_test)
```

#导入测试集, 从接口中调用需要的信息

决策树分类器DecisionTreeClassifier

```
class sklearn.tree.DecisionTreeClassifier (criterion='gini', splitter='best', max_depth=None,  
min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=None,  
random_state=None, max_leaf_nodes=None, min_impurity_decrease=0.0, min_impurity_split=None,  
class_weight=None, presort=False)
```

- 重要参数

- 为了要将表格转化为一棵树，决策树需要找出最佳节点和最佳的分枝方法，对分类树来说，衡量这个“最佳”的指标叫做“不纯度”。
- 通常来说，不纯度越低，决策树对训练集的拟合越好。现在使用的决策树算法在分枝方法上的核心大多是围绕在对某个不纯度相关指标的最优化上。
- 不纯度基于节点来计算，树中的每个节点都会有一个不纯度，并且子节点的不纯度一定是低于父节点的，也就是说，在同一棵决策树上，叶子节点的不纯度一定是最低的。
- Criterion这个参数正是用来决定不纯度的计算方法的。sklearn提供了两种选择：
 - 1) 输入“entropy”，使用信息熵（Entropy）
 - 2) 输入“gini”，使用基尼系数（Gini Impurity）

$$Entropy(t) = - \sum_{i=0}^{c-1} p(i|t) \log_2 p(i|t)$$

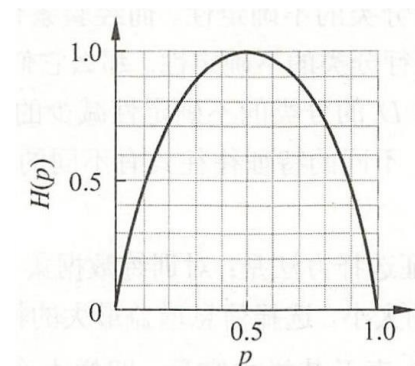
$$Gini(t) = 1 - \sum_{i=0}^{c-1} p(i|t)^2$$

信息增益计算方式

- 其中 t 代表给定的节点， i 代表标签的任意分类， $p(i|t)$ 代表标签分类 i 在节点 t 上所占的比例。注意，当使用信息熵时，`sklearn`实际计算的是基于信息熵的信息增益(Information Gain)，即父节点的信息熵和子节点的信息熵之差。
- 信息熵 (Entropy) :
 - 即0-1分布时，熵与概率的关系。当 $p=0$ 或 $p=1$ 时， $H(p)=0$ ，随机变量完全没有不确定性。当 $p=0.5$ 时，熵取值最大，随机变量不确定性最大。
- 基尼系数 (Gini Impurity) :
 - 其同样以上述熵的二分类例子为例，当两类数量相等时，基尼值等于0.5；当节点数据属于同一类时，基尼值等于0。基尼值越大，数据越不纯。

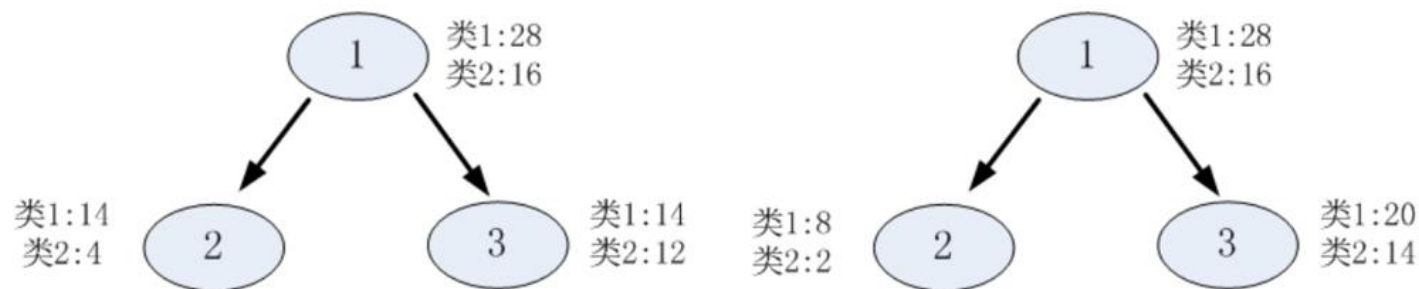
$$Entropy(t) = - \sum_{i=0}^{c-1} p(i|t) \log_2 p(i|t)$$

$$Gini(t) = 1 - \sum_{i=0}^{c-1} p(i|t)^2$$



信息增益计算方式

- 以熵作为节点复杂度的统计量，分别求出下面例子的信息增益，左图表示节点选择属性1进行分裂的结果，右图表示节点选择属性2进行分裂的结果，通过计算两个属性分裂后的信息增益，选择最优的分裂属性。



信息增益计算方式



$$Entropy(t) = - \sum_{i=0}^{c-1} p(i|t) \log_2 p(i|t)$$

$$Gini(t) = 1 - \sum_{i=0}^{c-1} p(i|t)^2$$

$$Info_1 = Entropy - \sum_{i=1}^n Entropy_i =$$

$$\frac{16}{28+16} \cdot \log\left(\frac{16}{28+16}\right) + \frac{28}{28+16} \cdot \log\left(\frac{28}{28+16}\right) \Rightarrow Entropy$$

$$-\left(\frac{14}{14+4} \cdot \log\left(\frac{14}{14+4}\right) + \frac{4}{14+4} \cdot \log\left(\frac{4}{14+4}\right)\right) \Rightarrow Entropy1$$

$$-\left(\frac{14}{14+12} \cdot \log\left(\frac{14}{14+12}\right) + \frac{12}{14+12} \cdot \log\left(\frac{12}{14+12}\right)\right) \Rightarrow Entropy2$$

$$= 0.81$$

$$Info_2 = Entropy - \sum_{i=1}^n Entropy_i =$$

$$\frac{16}{28+16} \cdot \log\left(\frac{16}{28+16}\right) + \frac{28}{28+16} \cdot \log\left(\frac{28}{28+16}\right) \Rightarrow Entropy$$

$$-\left(\frac{8}{8+2} \cdot \log\left(\frac{8}{8+2}\right) + \frac{2}{8+2} \cdot \log\left(\frac{2}{8+2}\right)\right) \Rightarrow Entropy1$$

$$-\left(\frac{20}{20+14} \cdot \log\left(\frac{20}{20+14}\right) + \frac{14}{20+14} \cdot \log\left(\frac{14}{20+14}\right)\right) \Rightarrow Entropy2$$

$$= 0.75$$

- $Info1 > Info2$, 所以属性1与属性2相比是更优的分裂属性, 选择属性1作为分裂属性。

信息增益计算方式

- 比起基尼系数，信息熵对不纯度更加敏感，对不纯度的惩罚最强。但是在实际使用中，信息熵和基尼系数的效果基本相同。信息熵的计算比基尼系数缓慢一些，因为基尼系数的计算不涉及对数。
- 另外，因为信息熵对不纯度更加敏感，所以信息熵作为指标时，决策树的生长会更加“精细”，因此对于高维数据或者噪音很多的数据，信息熵很容易过拟合，基尼系数在这种情况下效果往往比较好。当模型拟合程度不足的时候，即当模型在训练集和测试集上都表现不太好的时候，使用信息熵。
- 总结：基尼系数更加“温和”。

$$Entropy(t) = - \sum_{i=0}^{c-1} p(i|t) \log_2 p(i|t)$$

$$Gini(t) = 1 - \sum_{i=0}^{c-1} p(i|t)^2$$

信息增益计算方式

- 如何影响模型？
 - 确定不纯度的计算方法，帮忙找出最佳节点和最佳分枝，不纯度越低，决策树对训练集的拟合越好
- 可能的输入有哪些？
 - 不填默认基尼系数，填写gini使用基尼系数，填写entropy使用信息增益
- 怎样选取参数？
 - 通常就使用基尼系数
 - 数据维度很大，噪音很大时使用基尼系数，更加“温和”
 - 维度低，数据比较清晰的时候，信息熵和基尼系数没区别
 - 当决策树的拟合程度不够的时候，使用信息熵
 - 两个都试试，不好就换另外一个

$$Entropy(t) = - \sum_{i=0}^{c-1} p(i|t) \log_2 p(i|t)$$

$$Gini(t) = 1 - \sum_{i=0}^{c-1} p(i|t)^2$$

信息增益计算方式

- 决策树的基本流程其实可以简单概括如下：
 - 1、计算全部特征的不纯度指标
 - 2、选取不纯度指标最优的特征来分枝
 - 3、在第一个特征的分枝下，计算全部特征的不纯度指标
 - 4、选取不纯度指标最优的特征继续分枝
- 直到没有更多的特征可用，或整体的不纯度指标已经最优，决策树就会停止生长。

建立一棵树

- 1、导入需要的算法库和模块

```
from sklearn import tree
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
```

建立一棵树

- 2、数据描述性统计

```
wine = load_wine()
```

```
wine.data.shape
```

```
wine.target
```

#如果wine是一张表, 应该长这样:

```
import pandas as pd
```

```
pd.concat([pd.DataFrame(wine.data),pd.DataFrame(wine.target)],axis=1)
```

```
wine.feature_names
```

```
wine.target_names
```

建立一棵树

- 3、训练集测试集划分

```
Xtrain, Xtest, Ytrain, Ytest = train_test_split(wine.data, wine.target, test_size=0.3)
```

```
Xtrain.shape
```

```
Xtest.shape
```

建立一棵树

- 4、建立模型（三步走）

```
clf = tree.DecisionTreeClassifier(criterion="entropy")  
clf = clf.fit(Xtrain, Ytrain)  
score = clf.score(Xtest, Ytest) #返回预测的准确度
```

```
score
```

建立一棵树

- 5、树的可视化展示

- feature_name = ['酒精','苹果酸','灰','灰的碱性','镁','总酚','类黄酮','非黄烷类酚类','花青素','颜色强度','色调','od280/od315稀释葡萄酒','脯氨酸']

```
feature_name = ['酒精', '苹果酸', '灰', '灰的碱性', '镁', '总酚', '类黄酮', '非黄烷类酚类', '花青素', '颜色强度', '色调', 'od280/od315稀释葡萄酒', '脯氨酸']
```

```
import graphviz
```

```
dot_data = tree.export_graphviz(clf
                                ,out_file = None
                                ,feature_names= feature_name
                                ,class_names=["wine1", "wine2", "wine3"]
                                ,filled=True
                                ,rounded=True
                                )
```

```
graph = graphviz.Source(dot_data)
graph
```

建立一棵树

- 6、特征重要性程度

```
#特征重要性  
clf.feature_importances_  
  
[*zip(feature_name,clf.feature_importances_)]
```

决策树的随机性

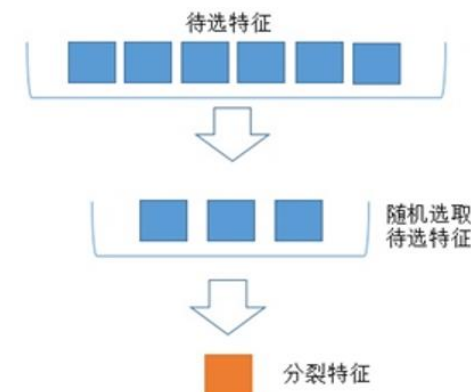
- 已经在只了解一个参数的情况下，建立了一棵完整的决策树。但是回到步骤4建立模型，`score`会在某个值附近波动，引起步骤5中画出来的每一棵树都不一样。它为什么会不稳定呢？如果使用固定数据集，它还会不稳定吗？
- 之前提到过，无论决策树模型如何进化，在分枝上的本质都还是追求某个不纯度相关的指标的优化，而正如我们提到的，不纯度是基于节点来计算的，也就是说，决策树在建树时，是靠优化节点来追求一棵优化的树，但最优的节点能够保证最优的树吗？
- 集成算法被用来解决这个问题：在`sklearn`中，既然一棵树不能保证最优，那就建更多的不同的树，然后从中取最好的。怎样从一组数据集中建不同的树？在每次分枝时，不使用全部特征，而是随机选取一部分特征，从中选取不纯度相关指标最优的作为分枝用的节点。这样，每次生成的树也就不同了。

```
clf = tree.DecisionTreeClassifier(criterion="entropy", random_state=30)
clf = clf.fit(Xtrain, Ytrain)
score = clf.score(Xtest, Ytest) #返回预测的准确度
```

score

重要参数random_state & splitter

- Sklearn为了保证树的随机性，不让某个坏特征（黄色都是猫，白色都是狗）影响最后结果，会随机选择特征建立树。
- splitter也是用来控制决策树中的随机选项的，有两种输入值，输入“best”，决策树在分枝时虽然随机，但是还是会优先选择更重要的特征进行分枝（重要性可以通过属性feature_importances_查看），输入“random”，决策树在分枝时更加随机，树会因为含有更多的不必要信息而更深更大，并因这些不必要信息而降低对训练集的拟合。
- 这也是防止过拟合的一种方式。当预测到模型会过拟合，用这两个参数来帮助你降低树建成之后过拟合的可能性。
- 当然，树一旦建成，依然主要是使用剪枝参数来防止过拟合。



重要参数random_state & splitter

```
clf = tree.DecisionTreeClassifier(criterion="entropy"  
                                ,random_state=30  
                                ,splitter="random"  
                                )  
  
clf = clf.fit(Xtrain, Ytrain)  
score = clf.score(Xtest, Ytest)  
  
score  
  
import graphviz  
dot_data = tree.export_graphviz(clf  
                                ,feature_names= feature_name  
                                ,class_names=["wine1", "wine2", "wine3"]  
                                ,filled=True  
                                ,rounded=True  
                                )  
  
graph = graphviz.Source(dot_data)  
graph
```

剪枝策略

- 在不加限制的情况下，一棵决策树会生长到衡量不纯度的指标最优，或者没有更多的特征可用为止。这样的决策树往往会过拟合，会在训练集上表现很好，在测试集上却表现糟糕。
- 收集的样本数据不可能和整体的状况完全一致，因此当一棵决策树对训练数据有了过于优秀的解释性，它找出的规则必然包含了训练样本中的噪声，并使它对未知数据的拟合程度不足。
- 为了让决策树有更好的泛化性，要对决策树进行剪枝。剪枝策略对决策树的影响巨大，正确的剪枝策略是优化决策树算法的核心。

剪枝参数max_depth

- max_depth限制树的最大深度，超过设定深度的树枝全部剪掉这是用得最广泛的剪枝参数，在高维度低样本量时非常有效。决策树多生长一层，对样本量的需求会增加一倍，所以限制树深度能够有效地限制过拟合。在集成算法中也非常实用。实际使用时，建议从=3开始尝试，看看拟合的效果再决定是否增加设定深度。

剪枝参数min_samples_leaf & min_samples_split

- min_samples_leaf限定，一个节点在分枝后的每个子节点都必须包含至少min_samples_leaf个训练样本，否则分枝就不会发生，在回归树中有神奇的效果，可以让模型变得更加平滑。
- 这个参数的数量设置得太小会引起过拟合，设置得太大就会阻止模型学习数据。一般来说，建议从=5开始使用。如果叶节点中含有的样本量变化很大，建议输入浮点数作为样本量的百分比来使用。同时，这个参数可以保证每个叶子的最小尺寸，可以在回归问题中避免低方差，过拟合的叶子节点出现。
- min_samples_split限定，一个节点必须要包含至少min_samples_split个训练样本，这个节点才允许被分枝，否则分枝就不会发生。

剪枝参数min_samples_leaf & min_samples_split

```
clf = tree.DecisionTreeClassifier(criterion="entropy"  
                                ,random_state=30  
                                ,splitter="random"  
                                ,max_depth=3  
                                ,min_samples_leaf=10  
                                ,min_samples_split=10  
                                )  
  
clf = clf.fit(Xtrain, Ytrain)  
  
dot_data = tree.export_graphviz(clf  
                                ,feature_names= feature_name  
                                ,class_names=["wine1", "wine2", "wine3"]  
                                ,filled=True  
                                ,rounded=True  
                                )  
  
graph = graphviz.Source(dot_data)  
graph  
  
clf.score(Xtrain,Ytrain)  
clf.score(Xtest,Ytest)
```

剪枝参数max_features & min_impurity_decrease

- 一般max_depth使用，用作树的“精修”
- max_features限制分枝时考虑的特征个数，超过限制个数的特征都会被舍弃。和max_depth异曲同工，max_features是用来限制高维度数据的过拟合的剪枝参数，但其方法比较暴力，是直接限制可以使用的特征数量而强行使决策树停下的参数，在不知道决策树中的各个特征的重要性的情况下，强行设定这个参数可能会导致模型学习不足。如果希望通过降维的方式防止过拟合，建议使用PCA，ICA或者特征选择模块中的降维算法。
- min_impurity_decrease限制信息增益的大小，信息增益小于设定数值的分枝不会发生。这是在0.19版本中更新的功能，在0.19版本之前时使用min_impurity_split。

如何确认最优的剪枝参数

- 那具体怎么来确定每个参数填写什么值呢？这时候，就要使用确定超参数的曲线来进行判断了，继续使用已经训练好的决策树模型`clf`。超参数的学习曲线，是一条以超参数的取值为横坐标，模型的度量指标为纵坐标的曲线，它是用来衡量不同超参数取值下模型的表现的线。在建好的决策树里，模型度量指标就是`score`。

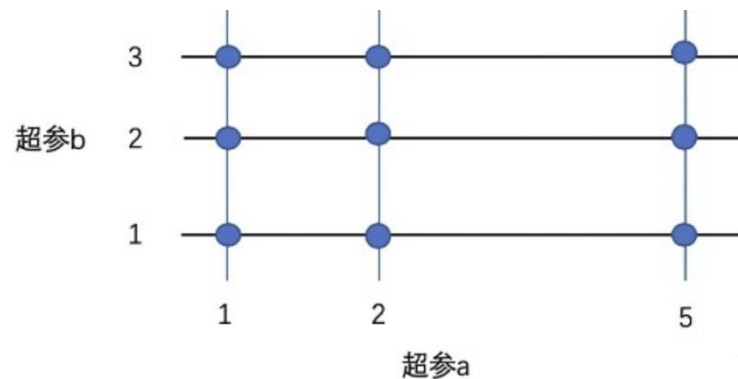
```
import matplotlib.pyplot as plt

test = []
for i in range(10):
    clf = tree.DecisionTreeClassifier(max_depth=i+1
                                     ,criterion="entropy"
                                     ,random_state=30
                                     ,splitter="random"
                                     )

    clf = clf.fit(Xtrain, Ytrain)
    score = clf.score(Xtest, Ytest)
    test.append(score)
plt.plot(range(1,11),test,color="red",label="max_depth")
plt.legend()
plt.show()
```

如何确认最优的剪枝参数

- 思考：
- 1、剪枝参数一定能够提升模型在测试集上的表现吗？
 - 调参没有绝对的答案，一切都是看数据本身。
- 2、这么多参数，一个个画学习曲线？
 - 需要用到网格搜索。泰坦尼克号案例中，会用代码来解答这个问题。
- 无论如何，剪枝参数的默认值会让树无尽地生长，这些树在某些数据集上可能非常巨大，对内存的消耗也非常巨大。所以如果你手中的数据集非常巨大，你已经预测到无论如何你都是要剪枝的，那提前设定这些参数来控制树的复杂性和大小会比较好。→预剪枝



非平衡样本的处理——class_weight

- 完成样本标签平衡的参数。样本不平衡是指在一组数据集中，标签的一类天生占有很大的比例。比如说，在银行要判断“一个办了信用卡的人是否会违约”，就是是vs否（1%：99%）的比例。这种分类状况下，即便模型什么也不做，全把结果预测成“否”，正确率也能有99%。因此要使用class_weight参数对样本标签进行一定的均衡，给少量的标签更多的权重，让模型更偏向少数类，向捕获少数类的方向建模。该参数默认None，此模式表示自动给与数据集中的所有标签相同的权重。
- 怎么评价分类算法的好坏，仅仅只看预测对了多少，这样做科学吗？

	预测结果	
真实情况	正例	反例
正例	TP（真正例）	FN（假反例）
反例	FP（假正例）	TN（真反例）

$$P = \frac{TP}{TP + FP}$$

$$R = \frac{TP}{TP + FN}$$

重要属性和方法

- 属性是在模型训练之后，能够调用查看的模型的各种性质。对决策树来说，最重要的是`feature_importances_`，能够查看各个特征对模型的重要性。
- `sklearn`中许多算法的接口都是相似的，比如说我们之前已经用到的`fit`和`score`，几乎对每个算法都可以使用。除了这两个接口之外，决策树最常用的接口还有`apply`和`predict`。`apply`中输入测试集返回每个测试样本所在的叶子节点的索引，`predict`输入测试集返回每个测试样本的标签。

```
#apply返回每个测试样本所在的叶子节点的索引  
clf.apply(Xtest)
```

```
#predict返回每个测试样本的分类/回归结果  
clf.predict(Xtest)
```

- 注意：所有接口中要求输入`X_train`和`X_test`的部分，输入的特征矩阵必须至少是一个二维矩阵。
- `sklearn`不接受任何一维矩阵作为特征矩阵被输入。如果你的数据的确只有一个特征，那必须用`reshape(-1,1)`来给矩阵增维；如果你的数据只有一个特征和一个样本，使用`reshape(1,-1)`来给你的数据增维。

决策树回归器——回归树

```
class sklearn.tree.DecisionTreeRegressor (criterion='mse', splitter='best', max_depth=None,  
min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=None,  
random_state=None, max_leaf_nodes=None, min_impurity_decrease=0.0, min_impurity_split=None, presort=False)
```

- 几乎所有参数，属性及接口都和分类树一模一样。需要注意的是，在回归树种，没有标签分布是否均衡的问题，因此没有class_weight这样的参数。

决策树回归器——回归树

```
class sklearn.tree.DecisionTreeRegressor (criterion='mse', splitter='best', max_depth=None,  
min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=None,  
random_state=None, max_leaf_nodes=None, min_impurity_decrease=0.0, min_impurity_split=None, presort=False)
```

- 回归树衡量分枝质量的指标，支持的标准有三种：
- 1、输入 “mse”使用均方误差mean squared error(MSE)，父节点和叶子节点之间的均方误差的差额将被用来作为特征选择的标准，这种方法通过使用叶子节点的均值来最小化L2损失。

$$MSE = \frac{1}{N} \sum_{i=1}^N (f_i - y_i)^2$$

- 2、输入 “friedman_mse”使用费尔德曼均方误差，这种指标使用弗里德曼针对潜在分枝中的问题改进后的均方误差。
- 3、输入 “mae”使用绝对平均误差MAE（mean absolute error），这种指标使用叶节点的中值来最小化L1损失。

决策树回归器——回归树

$$MSE = \frac{1}{N} \sum_{i=1}^N (f_i - y_i)^2$$

- 其中N是样本数量，i是每一个数据样本， f_i 是模型回归出的数值， y_i 是样本点i实际的数值标签。所以MSE的本质是样本真实数据与回归结果的差异。
- 在回归树中，MSE不只是我们的分枝质量衡量指标，也是最常用的衡量回归树回归质量的指标，当我们在使用交叉验证，或者其他方式获取回归树的结果时，我们往往选择均方误差作为评估（在分类树中这个指标是score代表的预测准确率）。在回归中追求的是MSE越小越好。
- 注意：虽然均方误差永远为正，但是sklearn当中使用均方误差作为评判标准时，却是计算“负均方误差”（`neg_mean_squared_error`）。
- 这是因为sklearn在计算模型评估指标的时候，会考虑指标本身的性质，均方误差本身是一种误差，所以被sklearn划分为模型的一种损失(loss)，因此在sklearn当中，都以负数表示。真正的均方误差MSE的数值，其实就是`neg_mean_squared_error`去掉负号的数字。

回归类模型的评价指标

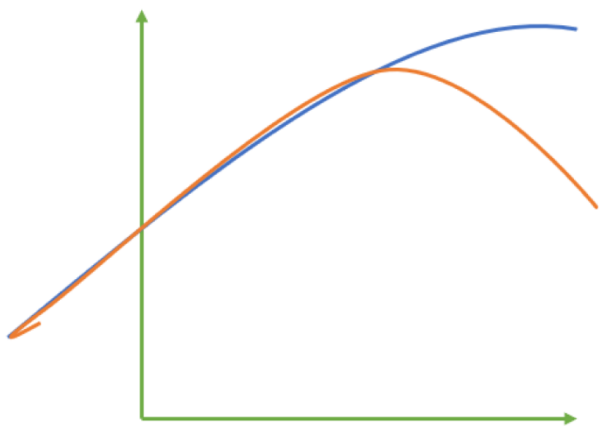
- 然而，回归树的接口`score`返回的是R平方，并不是MSE。
- 在我们的回归类算法中，我们有两种不同的角度来看待回归的效果：
- 第一，我们是否预测到了正确的数值。MSE、MAE。
- 第二，我们是否拟合到了足够的信息。R-Square。
- 这两种角度，分别对应着不同的模型评估指标。
- 注意和分类模型的区分：在分类型算法中，只有一种角度来评判，那就是是否预测到了正确的分类。

$$MSE = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

$$MAE = \frac{1}{m} \sum_{i=0}^{m-1} |y_i - \hat{y}_i|$$

回归类模型的评价指标

- 对于回归类算法而言，只探索数据预测是否准确是不足够的。除了数据本身的数值大小之外，还希望模型能够捕捉到数据的“规律”，而是否捕获了这些信息并无法使用MSE、MAE来衡量。
- 蓝色线是真实标签，而橙色线是拟合模型。对于这样的一个拟合模型，如果使用MSE来对它进行判断，它的MSE会很小，因为大部分样本其实都被完美拟合了，少数样本的真实值和预测值的巨大差异在被均分到每个样本上之后，MSE就会很小。
- 但这样的拟合结果必然不是一个好结果，因为一旦新样本是处于拟合曲线的后半段的，预测结果必然会有巨大的偏差，而这不是我们希望看到的。所以，我们希望找到新的指标，除了判断预测的数值是否正确之外，还能够判断我们的模型是否拟合了足够多的，数值之外的信息。



回归类模型的评价指标

- 方差=数据变异性=信息量，如果方差越大，代表数据上的信息量越多。
- 分子是真实值和预测值之差的差值，也就是我们的模型没有捕获到的信息总量。
- 分母是真实标签所带的信息量（Y的方差），所以其衡量的是1 - 模型没有捕获到的信息量占真实标签中所带的信息量的比例，所以，R-square越接近1越好。

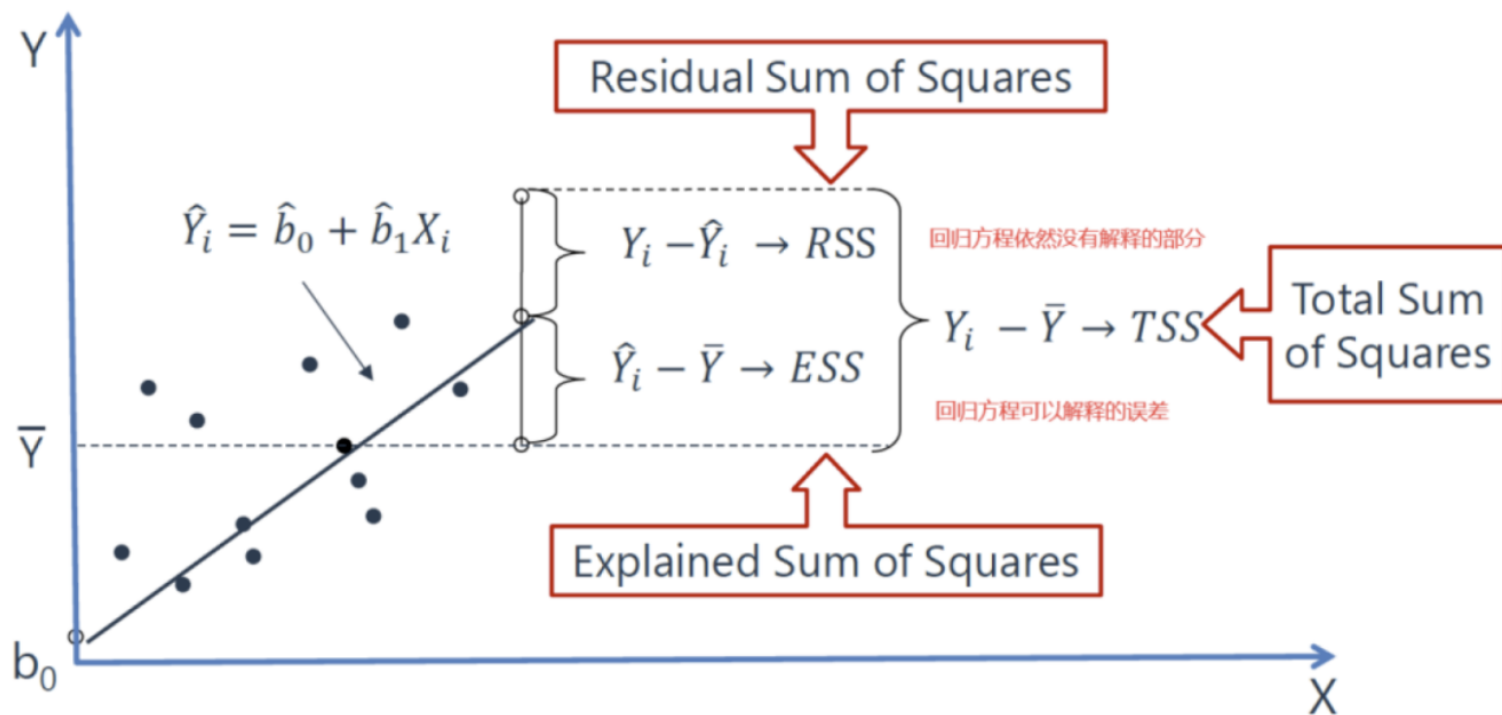
$$R^2 = 1 - \frac{\sum_{i=0}^m (y_i - \hat{y}_i)^2}{\sum_{i=0}^m (y_i - \bar{y})^2} = 1 - \frac{RSS}{\sum_{i=0}^m (y_i - \bar{y})^2}$$

回归类模型的评价指标

$$R^2 = 1 - \frac{\sum_{i=0}^m (y_i - \hat{y}_i)^2}{\sum_{i=0}^m (y_i - \bar{y})^2} = 1 - \frac{RSS}{TSS}$$

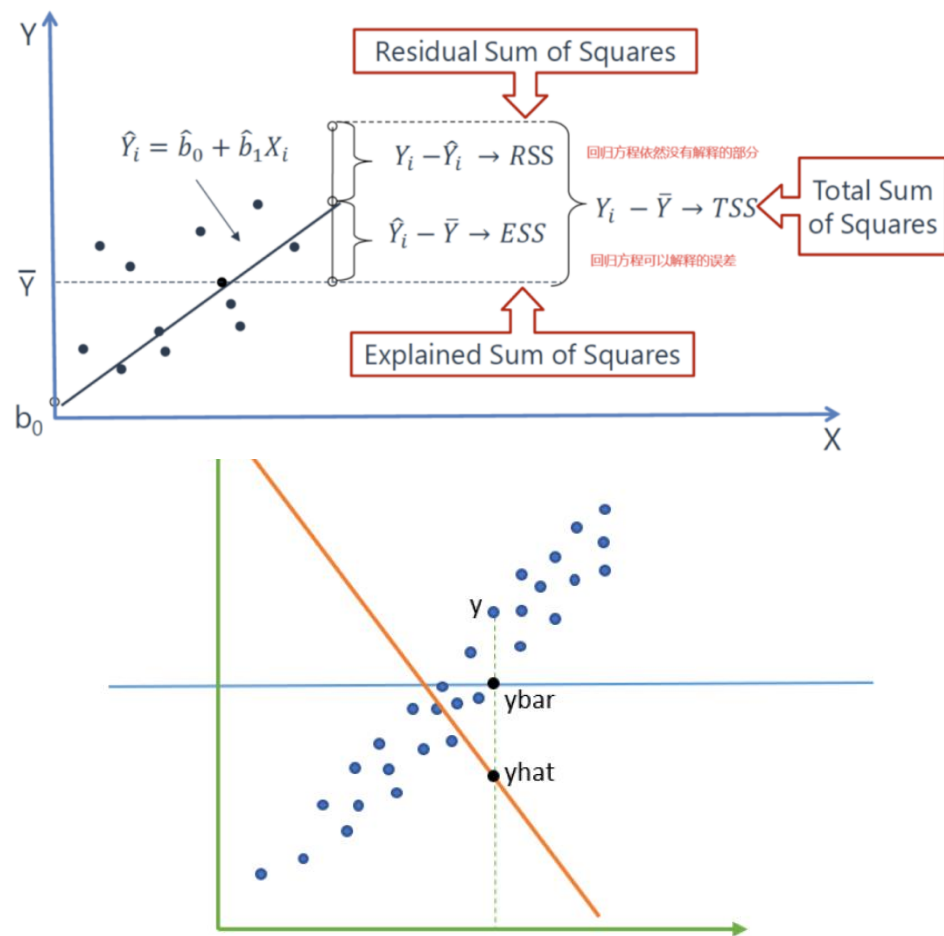
$$TSS = \sum_{i=0}^m (y_i - \bar{y})^2$$

$$ESS = \sum_{i=0}^m (\hat{y}_i - \bar{y})^2$$



回归类模型的评价指标

$$\begin{aligned}TSS &= \sum_{i=0}^m (y_i - \bar{y})^2 \\&= \sum_{i=0}^m (y_i - \hat{y}_i + \hat{y}_i - \bar{y})^2 \\&= \sum_{i=0}^m (y_i - \hat{y}_i)^2 + \sum_{i=0}^m (\hat{y}_i - \bar{y})^2 + 2 \sum_{i=0}^m (y_i - \hat{y}_i)(\hat{y}_i - \bar{y}) \\&= RSS + ESS + 2 \sum_{i=0}^m (y_i - \hat{y}_i)(\hat{y}_i - \bar{y}) \\1 &= \frac{RSS}{TSS} + \frac{ESS}{TSS} + \frac{2 \sum_{i=0}^m (y_i - \hat{y}_i)(\hat{y}_i - \bar{y})}{TSS} \\1 - \frac{RSS}{TSS} &= \frac{ESS + 2 \sum_{i=0}^m (y_i - \hat{y}_i)(\hat{y}_i - \bar{y})}{TSS} \\R^2 &= \frac{ESS + 2 \sum_{i=0}^m (y_i - \hat{y}_i)(\hat{y}_i - \bar{y})}{TSS}\end{aligned}$$



- 蓝色的横线是均值线，橙色的线是模型，蓝色的点是样本点。现在对于来说，真实标签减预测值的值为正，但预测值却是一个负数，这说明，数据本身的均值，比对数据的拟合模型本身更接近数据的真实值，那模型是完全没有作用的，类似于分类模型中的分类准确率小于50%，不如瞎猜。

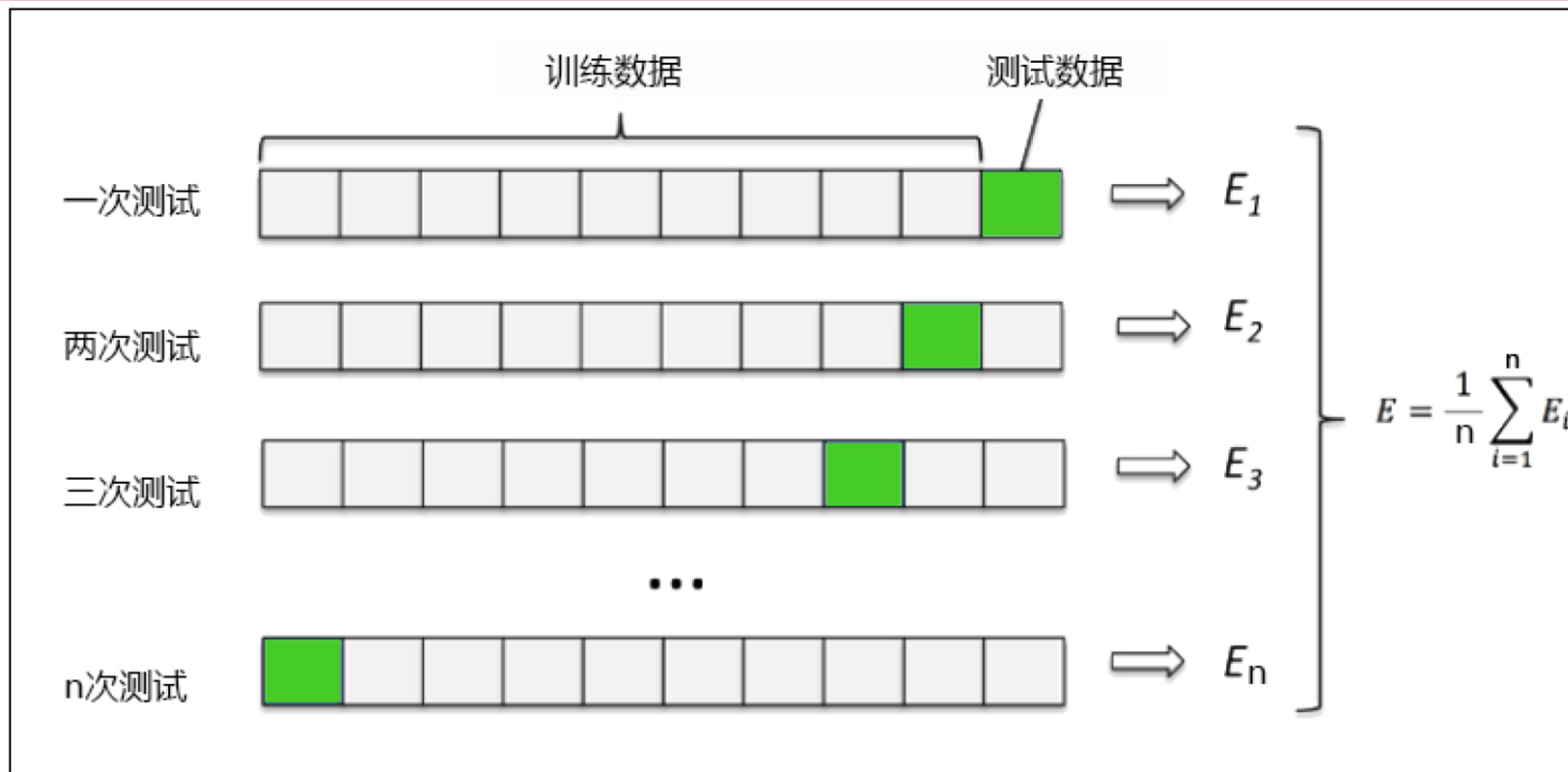
回归树

```
from sklearn.datasets import load_boston
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeRegressor

boston = load_boston()
regressor = DecisionTreeRegressor(random_state=0)
cross_val_score(regressor, boston.data, boston.target, cv=10,
                 scoring = "neg_mean_squared_error")
```

#交叉验证cross_val_score的用法

交叉验证



- 交叉验证是用来观察模型的稳定性的一种方法，我们将数据划分为n份，依次使用其中一份作为测试集，其他n-1份作为训练集，多次计算模型的精确性来评估模型的平均准确程度。训练集和测试集的划分会干扰模型的结果，因此用交叉验证n次的结果求出的平均值，是对模型效果的一个更好的度量。

决策树优点

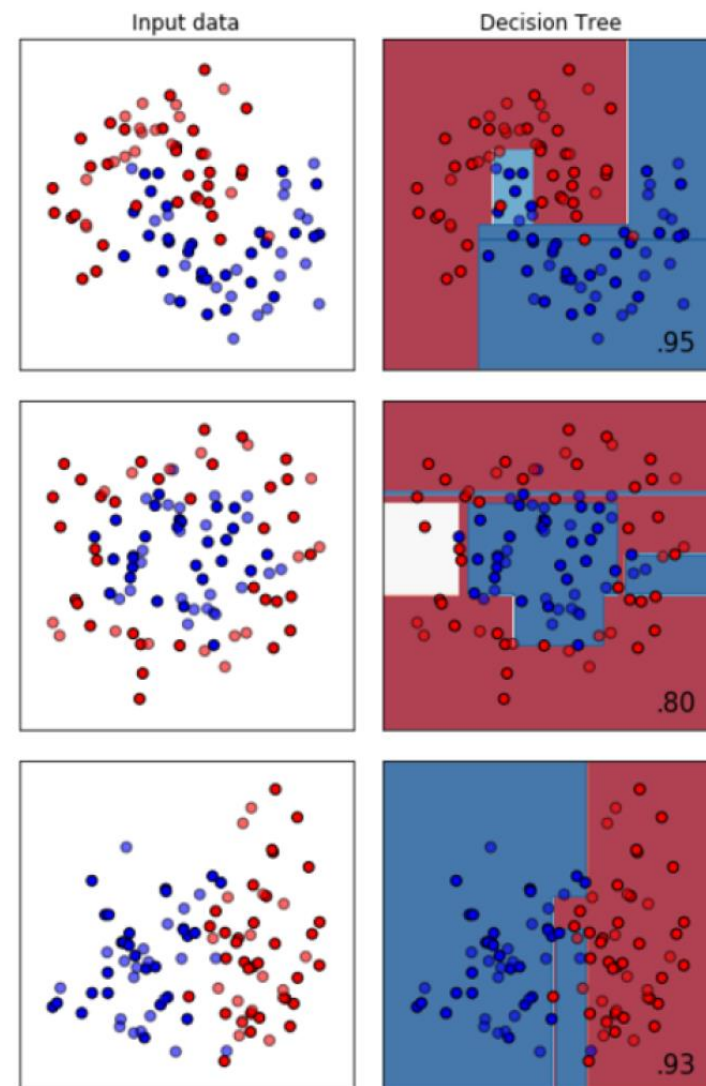
- 1、易于理解和解释，因为树木可以画出来被看见。
- 2、需要很少的数据准备。其他很多算法通常都需要数据规范化，需要创建虚拟变量并删除空值等。
但请注意，`sklearn`中的决策树模块不支持对缺失值的处理。
- 3、使用树的成本（比如说，在预测数据的时候）是用于训练树的数据点的数量的对数，相比于其他算法，这是一个很低的成本。
- 4、能够同时处理数字和分类数据，既可以做回归又可以做分类。其他技术通常专门用于分析仅具有一种变量类型的数据集。
- 5、能够处理多输出问题，即含有多个标签的问题，注意与一个标签中含有多种标签分类的问题区别开。
- 6、是一个白盒模型，结果很容易能够被解释。如果在模型中可以观察到给定的情况，则可以通过布尔逻辑轻松解释条件。相反，在黑盒模型中（例如，在人工神经网络中），结果可能更难以解释。

决策树缺点

- 1、决策树学习者可能创建过于复杂的树，这些树不能很好地推广数据，称为过度拟合。修剪（设置叶节点所需的最小样本数或设置树的最大深度）机制是避免此问题所必需的，而这些参数的整合和调整对初学者来说会比较晦涩。
- 2、决策树可能不稳定，数据中微小的变化可能导致生成完全不同的树，这个问题需要通过集成算法来解决。
- 3、决策树的学习是基于贪婪算法，它靠优化局部最优（每个节点的最优）来试图达到整体的最优，但这种做法不能保证返回全局最优决策树。这个问题也可以由集成算法来解决，在随机森林中，特征和样本会在分枝过程中被随机采样。
- 4、如果标签中的某些类占主导地位，决策树学习者会创建偏向主导类的树。因此，建议在拟合决策树之前平衡数据集。

巧妇难为无米之炊

- 从图上来看，每一条线都是决策树在二维平面上画出的一条决策边界，每当决策树分枝一次，就有一条线出现。当数据的维度更高的时候，这条决策边界就会由线变成面，甚至变成我们想象不出的多维图形。
- 同时，很容易看得出，分类树天生不擅长环形数据。每个模型都有自己的决策上限，所以一个怎样调整都无法提升表现的可能性也是有的。当一个模型怎么调整都不行的时候，可以选择换其他的模型使用。
- 最擅长月亮型数据的是最近邻算法，RBF支持向量机和高斯过程；最擅长环形数据的是最近邻算法和高斯过程；最擅长对半分的数据的是朴素贝叶斯，神经网络和随机森林。



总结

- 决策树是一种原理简单，应用广泛的模型，它可以同时被用于分类和回归问题。决策树的主要功能是从一张有特征和标签的表格中，通过对特定特征进行提问，为我们总结出一系列决策规则，并用树状图来呈现这些决策规则。
- 决策树的核心问题有两个，一个是如何找出正确的特征来进行提问，即如何分枝，二是树生长到什么时候应该停下。
- 对于第一个问题，我们定义了用来衡量分枝质量的指标不纯度，分类树的不纯度用基尼系数或信息熵来衡量，回归树的不纯度用MSE均方误差来衡量。每次分枝时，决策树对所有的特征进行不纯度计算，选取不纯度最低的特征进行分枝，分枝后，又再对被分枝的不同取值下，计算每个特征的不纯度，继续选取不纯度最低的特征进行分枝。

总结

- 每分枝一层，树整体的不纯度会越来越小，决策树追求的是最小不纯度。因此，决策树会一致分枝，直到没有更多的特征可用，或整体的不纯度指标已经最优，决策树就会停止生长。
- 决策树非常容易过拟合，这是说，它很容易在训练集上表现优秀，却在测试集上表现很糟糕。为了防止决策树的过拟合，我们要对决策树进行剪枝，`sklearn`中提供了大量的剪枝参数，

决策树在经济学中的应用

- 对于某些非线性和异质性情形，树模型比线性回归能得到更准确也更丰富的结论。
- 地铁对周边房价的影响，有11个特征，均为虚拟变量（地铁开通前后、是否靠近地铁、房屋大小、是否高层等等），地铁开通对房价的影响，跟房屋的其他特征密切相关，传统DID无法捕捉这一效应。
- Chin S, Kahn M E, Moon H R. Estimating the gains from new rail transit investment: A machine learning tree approach[J]. Real Estate Economics, 2020, 48(3): 886-914.

决策树在经济学中的应用

- Chin et al. (2018) 借助机器学习中的回归树算法，在双重差分法的框架下，讨论了首尔某条地铁开通对周边房地产市场的条件平均因果效应：在地铁周边住房的142个特征变量中，有89个特征会带来住房价格增值，53个特征会使得房价减值。

Appendix C. Conditional Average Treatment Effects

	CATE	S.E	N	Size (Category)	Room (No.)	Bath (No.)	Old (Category)	Other Transits Within 1km	District (Name)
1	1.0573	0.0509	201	Top 25%	3	2	< 5 yrs	Yes	Seocho
2	0.9023	0.0993	360	Bottom 25%	1	1	> 10yrs	Yes	Yeongdeungpo
3	0.8203	0.0968	96	Top 25%	5	2	5 ~ 10yrs	No	Yangcheon
4	0.6403	0.0780	208	Bottom 25% - 50%	3	1	5 ~ 10yrs	Yes	Seocho
5	0.5959	0.0442	1179	Bottom 25% - 50%	3	2	< 5 yrs	Yes	Seocho
6	0.5524	0.0630	536	Top 25% - 50%	4	2	< 5 yrs	Yes	Seocho
7	0.5453	0.0551	787	Top 25% - 50%	4	2	5 ~ 10yrs	Yes	Gangnam
8	0.5011	0.0726	1530	Bottom 25% - 50%	3	2	5 ~ 10yrs	No	Gangseo
9	0.4972	0.0941	628	Top 25%	4	2	5 ~ 10yrs	No	Gangseo
10	0.4637	0.0470	532	Bottom 25%	2	1	5 ~ 10yrs	Yes	Gangseo
11	0.4396	0.0911	1586	Top 25% - 50%	3	2	5 ~ 10yrs	No	Gangseo
12	0.4328	0.1324	228	Top 25%	3	2	5 ~ 10yrs	Yes	Seocho
13	0.4118	0.0283	757	Bottom 25%	3	1	5 ~ 10yrs	Yes	Gangseo

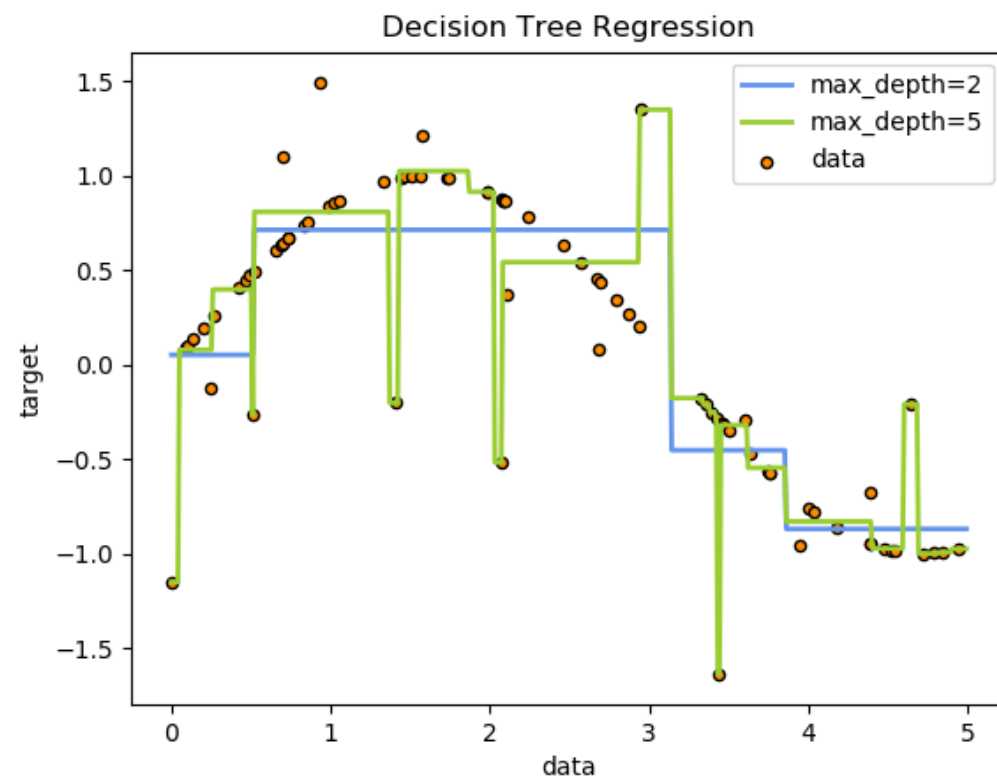
机器学习

2.2决策树（分类树、回归树）实战代码

吕斌

上海外国语大学国际金融贸易学院

案例一：回归树拟合正弦曲线



案例二：泰坦尼克号沉没数据集

- 泰坦尼克号的沉没是世界上最严重的海难事故之一，今天我们通过分类树模型来预测一下哪些人可能成为幸存者。数据集<https://www.kaggle.com/c/titanic>，数据集会随着代码一起提供给大家，大家可以在下载页面拿到，或者到群中询问。数据集包含两个csv格式文件，data为我们接下来要使用的数据，test为kaggle提供的测试集。

变量	定义	说明
PassengerId	乘客编号	
Survived	存活状况	0表示死亡，1表示幸存
Pclass	客舱等级	船票上的客舱等级，1表示一等舱，2表示二等舱，3表示三等舱
Name	姓名	
Sex	性别	male表示男性，female表示女性
Age	年龄	
Sibsp	船上的兄弟姐妹数或配偶数	指同代直系亲属数，忽略未婚妻或未婚夫
Parch	船上的父母数或子女数	指不同代直系亲属数，如果小孩是和除了父母之外的人一起出行的，则Parch=0
Ticket	船票编号	
Fare	船票价格	
Cabin	客舱号	
Embarked	登船港口	出发点是S表示的英国南安普顿，途经C表示的法国卢森堡以及Q代表的爱尔兰昆士敦

案例二：泰坦尼克号沉没数据集

